Computational Photography Pipeline on iPhone

Sreshth Tiwari
Dartmouth College
sreshth.tiwari.28@dartmouth.edu

Abstract

This project presents a mobile computational photography pipeline for iPhone that combines blur analysis, sharpening-based restoration, portrait-style background bokeh, tone mapping, and scene enhancement. The application is implemented as a hybrid React Native and Swift system, with native iOS modules handling image processing through Apple's Vision, Core Image, and AVFoundation frameworks. To keep the system practical for on-device execution, the project uses lightweight classical image processing methods rather than a deep learning model. Blur is estimated using Laplacian variance, while restoration is performed with sharpening filters to improve mildly blurred images. Additional modules apply synthetic bokeh, global tone adjustment, and heuristic scene enhancement to improve the overall appearance of captured images. Results show that the app successfully demonstrates an end-to-end computational photography workflow on mobile hardware, providing a modular baseline for future extensions such as true motion deblurring and improved scene understanding.

1. Introduction

Smartphone cameras increasingly rely on computational photography to compensate for the limitations of small lenses and sensors. Features such as portrait mode, tone correction, and blur reduction are now standard on modern phones, but they require a combination of image processing and scene understanding. This project investigates how these techniques can be implemented on iPhone hardware using native Apple frameworks.

The goal of the project was to build a mobile application that could analyze image sharpness, enhance portraits with synthetic background blur, and apply several forms of image correction including sharpening, tone mapping, and scene-dependent enhancement. To keep the system practical for mobile deployment, classical image processing methods were used rather than a large deep learning model. The final application was implemented in React Native for the user interface, with Swift modules handling the image-processing pipeline on iOS.

2. Related Work

Computational photography covers a broad range of image enhancement techniques, ranging from HDR tone mapping to segmentation-based portrait mode and deblurring [7]. Apple's portrait mode combines subject segmentation or depth estimation with background blur synthesis to simulate shallow depth of field [3]. Motion blur reduction has also been studied extensively. Methods include simple sharpening filters as well as iterative deconvolution and learned restoration models [6, 4].

For this project, Apple's Vision framework was used for person segmentation, Core Image for filtering and compositing, and AVFoundation for camera capture [3, 2, 1]. Classical blur estimation methods such as Laplacian variance were also referenced, since they provide a simple measure of sharpness by examining high-frequency image content [5].

3. System Overview

The application is organized as a hybrid React Native and Swift pipeline. React Native provides the interface, and Swift modules implement the image-processing operations. The current system operates on captured images and supports independent processing steps as well as a full pipeline mode.

The main pipeline components are:

- **Blur analysis:** compute a sharpness score using Laplacian variance.
- **Restoration:** apply lightweight sharpening filters to reduce mild blur.
- **Bokeh:** segment the person and blur the background.
- **Tone mapping:** adjust exposure, shadows, highlights, and contrast.
- **Scene enhancement:** apply simple scene-dependent filter tuning.

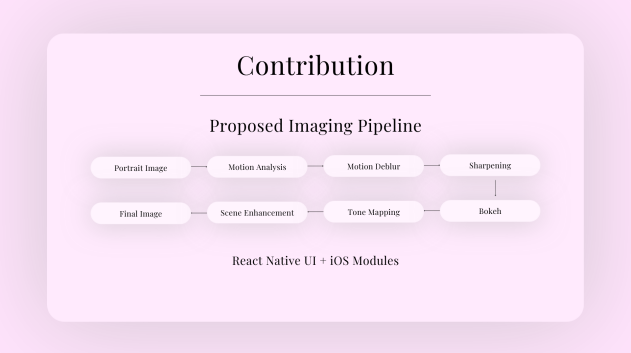

Figure 1. System overview. The app loads an input image, estimates blur, applies restoration or enhancement filters, and outputs a processed result for side-by-side comparison.

4. Methods

4.1. Motion Blur Analysis

To estimate blur, the image is converted to grayscale and the variance of the Laplacian response is measured. Sharp images tend to have more strong edges, which produces a higher variance. Blurry images suppress high-frequency detail and therefore give a lower score. This metric is lightweight and works well as a baseline for blur detection on mobile hardware [5].

4.2. Restoration by Sharpening

Instead of implementing a full deblurring model, classical sharpening filters were used as a baseline restoration method. The app applies the Core Image filters `CISharpenLuminance` and `CIUnsharpMask` to increase local edge contrast. These methods do not reconstruct the original sharp image, but they reduce the appearance of mild blur and improve visual clarity with minimal runtime cost. They were chosen due to their relative runtime savings compared to traditional restoration methods like Richardson-Lucy deconvolution [6, 4].

4.3. Background Bokeh

For the portrait effect, Apple's Vision framework is used to generate a person segmentation matte. The subject is separated from the background, the background is blurred using Gaussian blur, and the two are composited together with Core Image. This produces a synthetic bokeh effect that mimics shallow depth of field [3, 2].

4.4. Tone Mapping

Tone mapping is implemented as a lightweight global adjustment pipeline using Core Image filters. The input image is first passed through `CIExposureAdjust` to shift the overall brightness of the scene. This provides a coarse luminance correction and helps compensate for images that are too dark or too bright. The image is then processed with `CIHighlightShadowAdjust`, which separately lifts shadow detail and compresses bright regions to reduce clipping in highlights. Finally, `CIColorControls` is used to fine-tune contrast, saturation, and brightness, producing a more balanced final result. Together, these operations approximate a simple tone-mapping workflow that improves global dynamic range and visual consistency without requiring an explicit HDR reconstruction step [2].

4.5. Scene Enhancement

Scene enhancement is implemented as a heuristic adjustment pipeline that applies a different combination of Core Image filters to improve the overall appearance of the image. The current implementation uses `CIExposureAdjust` to slightly brighten the image, `CIVibrance` to selectively increase color intensity in a more natural way than uniform saturation, and `CIColorControls` to adjust contrast and saturation after the initial exposure change. This approach is intended to improve dull or flat-looking images by making them appear more vivid and better balanced, while still keeping the processing lightweight enough for on-device execution [2].

5. Implementation

The application was built as a React Native iOS app with Swift native modules. React Native handles the user interface and button controls, while Swift handles the image-processing operations. The app uses:

- **AVFoundation** for camera capture
- **Vision** for person segmentation
- **Core Image** for filtering and compositing
- **React Native native modules** for bridging JavaScript and Swift

The project was structured so that each operation could be tested independently. This made it easier to debug the native pipeline and to compare outputs from blur analysis, sharpening, bokeh, tone mapping, and scene enhancement.

6. Results

The app successfully demonstrates an on-device computational photography pipeline. The blur analysis module provides a sharpness score that distinguishes clearer images from softer ones. The sharpening baseline improves mildly blurred images. The bokeh module can separate a subject and blur the background for portrait-style enhancement. Tone mapping and scene enhancement improve the image's overall appearance by adjusting global visual properties.

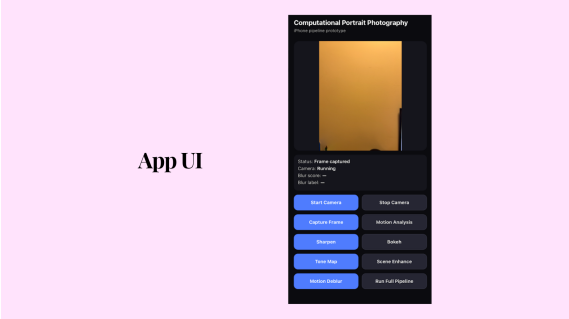

Figure 2. App UI. The app allows users to test each component individually or run the whole pipeline on the image.

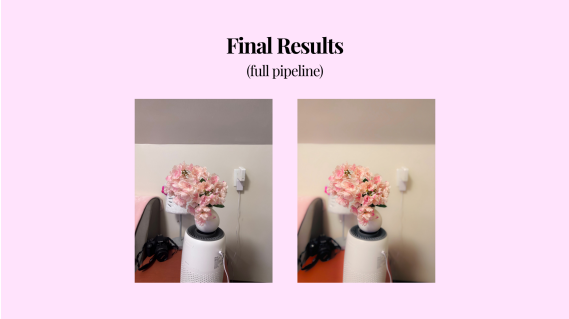

Figure 3. Before-and-after comparison of the portrait enhancement pipeline.

Module	Status
Blur analysis	Implemented
Sharpening restoration	Implemented
Bokeh / portrait blur	Implemented
Tone mapping	Implemented
Scene enhancement	Implemented
True motion deblurring	Not implemented

Table 1. Implementation status of major pipeline components.

7. Discussion

The current system demonstrates the tradeoff between quality and runtime in mobile image processing. The implemented filters are fast and practical on iPhone hardware, but they are not as powerful as more advanced deep learning or iterative deblurring methods [6, 4]. Specifically, sharpening-based restoration is only a baseline for motion blur reduction, and the bokeh pipeline depends on segmentation quality.

A major advantage of the chosen design is modularity. Each effect can be triggered independently, and a full pipeline button allows the user to process an image end-to-end. This structure makes the system useful both as a demo and as a foundation for future extension.

8. Limitations and Future Work

There are several limitations in the current prototype. First, the motion blur module only performs estimation and lightweight sharpening, and does not actually perform true blur kernel estimation and deconvolution. Second, segmentation quality can vary depending on the input image. Third, the scene enhancement and tone mapping steps are heuristic and do not perform semantic scene understanding.

Future work could include:

- true motion deblurring using kernel estimation and iterative deconvolution
- improved matte refinement for more accurate portrait boundaries
- more advanced scene classification
- better runtime optimization for live camera use

9. Conclusion

This project shows how computational photography techniques can be implemented on iPhone using a hybrid React Native and Swift architecture [3, 2, 1]. The current prototype demonstrates blur analysis, portrait-style bokeh, sharpening-based restoration, tone mapping, and scene enhancement on-device. Although the system does not yet include full motion deblurring, it provides a strong baseline pipeline for mobile image enhancement and a platform for future improvements.

References

[1] Apple Inc. Avfoundation framework documentation, 2024. Online documentation. 1, 3

[2] Apple Inc. Core image framework documentation, 2024. Online documentation. 1, 2, 3

[3] Apple Inc. Vision framework documentation, 2024. Online documentation. 1, 2, 3

[4] L. B. Lucy. An iterative technique for the rectification of observed distributions. *The Astronomical Journal*, 1974. 1, 2, 3

[5] Haibo Luo and Wenliang Zuo. A simple and effective focus measure for image blur detection. *Pattern Recognition*, 2011. 1, 2

[6] William H. Richardson. Bayesian-based iterative method of image restoration. *Journal of the Optical Society of America*, 1972. 1, 2, 3

[7] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2022. 1